WEEK 4 PROJECT

ORDER MANAGEMENT SYSTEM

Database Design, SQL CRUD Operations & Customer Purchase Analysis

Subject	Database Management System (MySQL)
Project	Week 4 – Order Management System
Database	ecommerce
Prepared for	Academic submission

This report includes the table design, relationships, sample data, SQL operations, and customer/product sales reports required for the Week 4 task.

1. Objective

To design an order-management database that connects customers, orders, order line items, and products. The database uses primary keys, foreign keys, NOT NULL, UNIQUE, and CHECK constraints, and supports CRUD operations and sales analysis.

2. Table Design

Customer

Column	Data type	Constraint / purpose
Customer_ID	INT	Primary key, auto-increment
Customer_Name	VARCHAR(100)	NOT NULL
Email	VARCHAR(150)	NOT NULL, UNIQUE
Phone	VARCHAR(15)	UNIQUE
City	VARCHAR(80)	NOT NULL

Product

Column	Data type	Constraint / purpose
Product_ID	INT	Primary key, auto-increment
Product_Name	VARCHAR(120)	NOT NULL, UNIQUE
Category	VARCHAR(80)	NOT NULL
Price	DECIMAL(10,2)	NOT NULL, CHECK Price >= 0
Stock	INT	NOT NULL, default 0, CHECK Stock >= 0

Orders

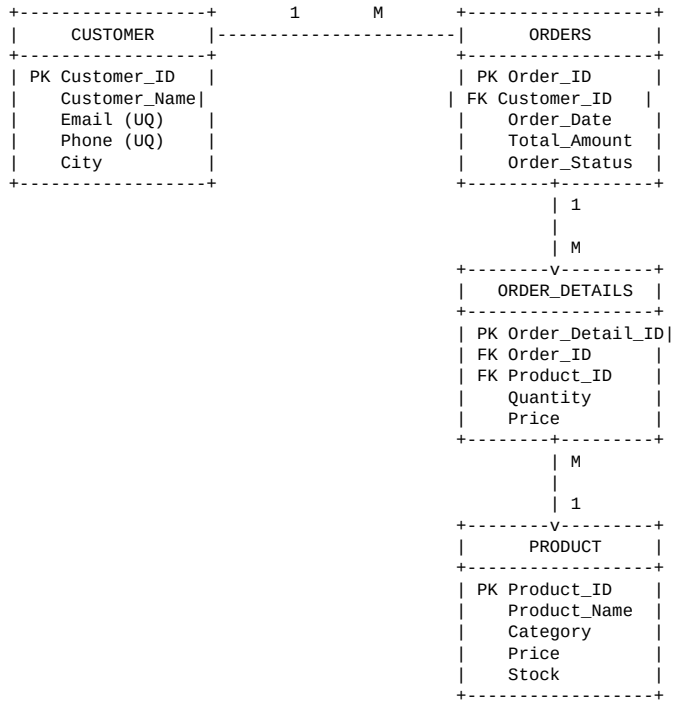
Column	Data type	Constraint / purpose
Order_ID	INT	Primary key, auto-increment
Customer_ID	INT	NOT NULL, FK → Customer.Customer_ID
Order_Date	DATE	NOT NULL
Total_Amount	DECIMAL(10,2)	NOT NULL, default 0, CHECK >= 0
Order_Status	VARCHAR(20)	NOT NULL; Pending/Shipped/Delivered/Cancelled

Order_Details

Column	Data type	Constraint / purpose
Order_Detail_ID	INT	Primary key, auto-increment
Order_ID	INT	NOT NULL, FK → Orders.Order_ID
Product_ID	INT	NOT NULL, FK → Product.Product_ID
Quantity	INT	NOT NULL, CHECK Quantity > 0
Price	DECIMAL(10,2)	NOT NULL, CHECK Price >= 0; unit price at order time
Combined constraint	UNIQUE	UNIQUE (Order_ID, Product_ID) prevents duplicate product lines per or

3. ER Diagram

The Customer-to-Orders relationship is one-to-many. Each order can contain multiple Order_Details rows, and each product can appear in many order-detail rows.



PK = Primary Key; **FK** = Foreign Key; **UQ** = Unique constraint.

4. Sample Data

Sample customers

Customer_ID	Customer_Name	Email	City
1	Arun Kumar	arun@example.com	Thoothukudi
2	Priya Devi	priya@example.com	Madurai
3	Karthik Raj	karthik@example.com	Tirunelveli
4	Meena S	meena@example.com	Chennai

Sample products

Product_ID	Product_Name	Category	Price (■)
101	Laptop	Electronics	45,000.00
102	Wireless Mouse	Accessories	650.00
103	Keyboard	Accessories	1,200.00
104	Headphones	Accessories	1,800.00
105	USB Drive 64GB	Storage	550.00

Sample orders

Order_ID	Customer_ID	Date	Total (■)	Status
1001	1	2026-10-01	46,300.00	Delivered

Order_ID	Customer_ID	Date	Total (■)	Status
1002	2	2026-10-03	3,000.00	Shipped
1003	1	2026-10-10	2,300.00	Pending
1004	3	2026-10-12	45,000.00	Delivered
1005	4	2026-10-15	1,750.00	Pending

5. Core SQL Operations

The complete executable MySQL script is supplied separately as Week4_Order_Management_System.sql. The following are the key operations included in that script.

Create a new order and add multiple products

```
INSERT INTO Orders (Customer_ID, Order_Date, Total_Amount, Order_Status)
VALUES (2, CURRENT_DATE(), 0.00, 'Pending');
SET @new_order_id = LAST_INSERT_ID();

INSERT INTO Order_Details (Order_ID, Product_ID, Quantity, Price)
VALUES (@new_order_id, 102, 2, 650.00),
       (@new_order_id, 105, 1, 550.00);

UPDATE Orders
SET Total_Amount = (
  SELECT COALESCE(SUM(Quantity * Price), 0)
  FROM Order_Details WHERE Order_ID = @new_order_id
)
WHERE Order_ID = @new_order_id;
```

Update status and quantity

```
UPDATE Orders SET Order_Status = 'Shipped' WHERE Order_ID = 1002;
UPDATE Orders SET Order_Status = 'Delivered' WHERE Order_ID = 1005;

UPDATE Order_Details
SET Quantity = 3
WHERE Order_ID = 1001 AND Product_ID = 102;

-- Recalculate order total after a quantity change
UPDATE Orders o
SET o.Total_Amount = (
  SELECT COALESCE(SUM(Quantity * Price), 0)
  FROM Order_Details WHERE Order_ID = o.Order_ID
)
WHERE o.Order_ID = 1001;
```

Display complete order details

```
SELECT o.Order_ID, c.Customer_Name, o.Order_Date, o.Order_Status,
       p.Product_Name, od.Quantity, od.Price,
       od.Quantity * od.Price AS Line_Total, o.Total_Amount
FROM Orders o
JOIN Customer c ON c.Customer_ID = o.Customer_ID
JOIN Order_Details od ON od.Order_ID = o.Order_ID
JOIN Product p ON p.Product_ID = od.Product_ID
ORDER BY o.Order_ID;
```

Delete a cancelled order

```
UPDATE Orders SET Order_Status = 'Cancelled' WHERE Order_ID = 1003;
DELETE FROM Orders
WHERE Order_ID = 1003 AND Order_Status = 'Cancelled';
```

Because Order_Details has ON DELETE CASCADE, its line items are removed automatically when the parent order is deleted. Product rows cannot be deleted while referenced by order details.

6. Customer Order History & Purchase Analysis

```
-- Customer order history
SELECT c.Customer_Name, o.Order_ID, o.Order_Date,
       o.Order_Status, o.Total_Amount
FROM Customer c JOIN Orders o ON c.Customer_ID = o.Customer_ID
ORDER BY c.Customer_ID, o.Order_Date;

-- Product-wise orders, quantity sold, and sales
SELECT p.Product_Name, COUNT(DISTINCT od.Order_ID) AS Orders_Containing_Product,
       SUM(od.Quantity) AS Total_Quantity_Sold,
       SUM(od.Quantity * od.Price) AS Product_Sales
FROM Product p
LEFT JOIN Order_Details od ON od.Product_ID = p.Product_ID
LEFT JOIN Orders o ON o.Order_ID = od.Order_ID
WHERE o.Order_Status <> 'Cancelled' OR o.Order_ID IS NULL
GROUP BY p.Product_ID, p.Product_Name;

-- Number of times each product was ordered
SELECT p.Product_Name, COUNT(od.Order_Detail_ID) AS Times_Ordered
FROM Product p
LEFT JOIN Order_Details od ON od.Product_ID = p.Product_ID
LEFT JOIN Orders o ON o.Order_ID = od.Order_ID
WHERE o.Order_Status <> 'Cancelled' OR o.Order_ID IS NULL
GROUP BY p.Product_ID, p.Product_Name;

-- Total quantity sold
```

```sql
SELECT SUM(od.Quantity) AS Total_Quantity_Sold
FROM Order_Details od JOIN Orders o ON o.Order_ID = od.Order_ID
WHERE o.Order_Status <> 'Cancelled';

-- Customer with the maximum number of orders
SELECT c.Customer_Name, COUNT(*) AS Number_Of_Orders
FROM Customer c JOIN Orders o ON o.Customer_ID = c.Customer_ID
WHERE o.Order_Status <> 'Cancelled'
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY Number_Of_Orders DESC;

-- Total spending by customer
SELECT c.Customer_Name,
       COALESCE(SUM(CASE WHEN o.Order_Status <> 'Cancelled'
                         THEN o.Total_Amount ELSE 0 END), 0) AS Total_Spending
FROM Customer c LEFT JOIN Orders o ON o.Customer_ID = c.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name;

-- Average order value and total sales
SELECT ROUND(AVG(Total_Amount), 2) AS Average_Order_Value
FROM Orders WHERE Order_Status <> 'Cancelled';

SELECT ROUND(SUM(Total_Amount), 2) AS Total_Sales
FROM Orders WHERE Order_Status <> 'Cancelled';
```

7. Illustrative Report Outputs

The tables below show expected results from the sample data before running the optional CRUD examples that change or delete sample orders. Run the report queries after loading the sample data to capture screenshots from MySQL Workbench.

Customer order history (sample)

Customer	Order_ID	Order Date	Status	Total (■)
Arun Kumar	1001	2026-10-01	Delivered	46,300.00
Arun Kumar	1003	2026-10-10	Pending	2,300.00
Priya Devi	1002	2026-10-03	Shipped	3,000.00
Karthik Raj	1004	2026-10-12	Delivered	45,000.00
Meena S	1005	2026-10-15	Pending	1,750.00

Purchase summary (sample)

Metric	Expected value
Number of sample orders	5
Total sample sales	■98,350.00
Average order value	■19,670.00
Customer with maximum orders	Arun Kumar (2 orders)

Note: These are illustrative outputs based on the five sample orders. The SQL file includes all required report queries. Screenshots should be captured after executing the queries in MySQL Workbench; screenshots cannot be generated truthfully without running them in your own database environment.

8. Constraints and Relationship Summary

Requirement	Implementation
Primary Key	Customer_ID, Product_ID, Order_ID, Order_Detail_ID
Foreign Key	Orders.Customer_ID → Customer.Customer_ID
Foreign Key	Order_Details.Order_ID → Orders.Order_ID
Foreign Key	Order_Details.Product_ID → Product.Product_ID
NOT NULL	Required customer, product, order, quantity, and price fields
UNIQUE	Customer Email, Phone, Product_Name, and (Order_ID, Product_ID)
CHECK	Non-negative prices/totals/stock, positive quantity, allowed order statuses

9. How to Run and Submit

1. Open MySQL Workbench and connect to your MySQL server.
2. Open Week4_Order_Management_System.sql and execute it. If your existing database already contains Customer or Product tables, compare their column names first to avoid table-definition conflicts.
3. Run the report queries individually and capture screenshots of the result grid.
4. Upload this PDF and the .sql script to your GitHub repository.
5. Submit the repository URL in the Google Form provided by your instructor.

Important: The script uses the database name **ecommerce** and assumes MySQL 8.0+ for CHECK constraint enforcement. The report is a prepared project document; add your own details and actual Workbench screenshots before final submission.